Versi 1.0

Agustus 2026

ALGORITMA PEMROGRAMAN

MODUL 3 - Merge Sort & Quick Sort

Author:

Nur Hayatin, S.ST., M.Kom. Ph.D

Muhammad Ibrahim Al Ayubi

Mareno Ali Ibrahim

INFORMATICS LABORATORY

University of Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI **2**
INTRODUCTION **3**
TUJUAN 3
CAPAIAN MODUL 3
PERSIAPAN 3
KEYWORDS 3
CONTENTS **4**
1. Konsep Recursive Sorting 4
1.1. Definisi Rekursi 4
1.2. Base Case dan Recursive Case 4
1.3. Konsep Divide and Conquer 5
2. Merge Sort 5
2.1. Definisi Merge Sort 5
2.2. Cara Kerja Merge Sort 6
2.3. Analisis Kompleksitas Waktu Merge Sort 11
2.4. Kelebihan dan Kekurangan Merge Sort 12
3. Quick Sort 13
3.1. Definisi Quick Sort 13
3.2. Cara Kerja Quick Sort 14
3.3. Analisis Kompleksitas Waktu Quick Sort 19
3.4. Kelebihan dan Kekurangan Quick Sort 19
CODELAB **22**
Task 1 - Merge Sort (Descending Order) 22
Task 2 - Quick Sort (Descending Order) 24
DEMO **26**
Penilaian **28**
Rubrik Penilaian 28
Rating Scale 28

INTRODUCTION

TUJUAN

1. Membantu mahasiswa memahami konsep dasar algoritma pengurutan dalam pemrograman.
2. Memperkenalkan cara data dapat diorganisasi menggunakan teknik Divide and Conquer.
3. Memberikan pemahaman praktis kepada mahasiswa tentang implementasi Merge Sort dan Quick Sort menggunakan Java.

CAPAIAN MODUL

1. Mahasiswa mampu memahami cara kerja Merge Sort dan Quick Sort.
2. Mahasiswa mampu memahami konsep sorting rekursif dalam pemrograman Java.
3. Mahasiswa mampu menganalisis efisiensi algoritma pengurutan menggunakan kompleksitas waktu (Big O).
4. Mahasiswa mampu membandingkan Merge Sort dan Quick Sort berdasarkan performa dan penggunaan memori.

PERSIAPAN

1. Mahasiswa Muslim dapat membaca doa berikut sebelum memulai pembelajaran:

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop atau komputer pribadi
3. IntelliJ IDEA/IDE Java lainnya (Eclipse/VSCode)
4. Motivasi dan komitmen yang kuat untuk belajar

KEYWORDS

Sorting, Merge Sort, Quick Sort, Divide and Conquer, Recursion, Pivot, Array, Time Complexity, Java Programming

CONTENTS

1. Konsep Recursive Sorting

Recursive sorting merupakan teknik pengurutan yang menggunakan konsep rekursi untuk menyelesaikan suatu masalah dengan membaginya menjadi beberapa submasalah yang lebih kecil. Pada algoritma pengurutan rekursif, sebuah fungsi akan memanggil dirinya sendiri hingga mencapai kondisi berhenti tertentu. Konsep ini umum digunakan pada algoritma pengurutan seperti Merge Sort dan Quick Sort karena memungkinkan masalah berukuran besar diselesaikan secara lebih efisien.

1.1. Definisi Rekursi

Rekursi merupakan teknik pemrograman ketika sebuah fungsi memanggil dirinya sendiri secara berulang untuk menyelesaikan versi yang lebih kecil dari masalah yang sama. Fungsi rekursif akan terus berjalan hingga mencapai kondisi berhenti yang disebut base case. Rekursi berguna untuk menyelesaikan masalah yang dapat dibagi menjadi submasalah yang lebih kecil dan serupa. Pada contoh berikut, fungsi memanggil dirinya sendiri dengan nilai yang semakin kecil hingga nilainya menjadi 0.

Recurvise

```
void countdown(int n) {
    if (n == 0) {
        return;
    }

    System.out.println(n);
    countdown(n - 1);
}
```

Gambar 1. *Contoh Rekursif*

1.2. Base Case dan Recursive Case

Sebuah fungsi rekursif terdiri dari dua bagian penting:

- **Base Case**
  Kondisi yang menghentikan proses rekursi. Tanpa base case, fungsi akan terus berjalan tanpa batas dan dapat menyebabkan error.
- **Recursive Case**
  Bagian ketika fungsi memanggil dirinya sendiri dengan masukan yang lebih kecil atau lebih sederhana.

```java
void printNumber(int n) {
    if (n == 0) { // Base Case
        return;
    }

    System.out.println(n);
    printNumber(n - 1); // Recursive Case
}
```

Gambar 2. *Base Case & Recursive Case*

1.3. Konsep Divide and Conquer

Divide and Conquer merupakan teknik penyelesaian masalah yang membagi masalah besar menjadi beberapa submasalah yang lebih kecil, menyelesaikan setiap submasalah, lalu menggabungkan hasilnya untuk membentuk solusi akhir. Konsep ini banyak digunakan dalam Merge Sort dan Quick Sort. Tahapan umum Divide and Conquer adalah:

1. Divide
   Membagi masalah menjadi beberapa bagian yang lebih kecil.
2. Conquer
   Menyelesaikan setiap sub-masalah secara rekursif.
3. Combine
   Menggabungkan hasil dari setiap submasalah menjadi satu solusi akhir.

Sebagai contoh, Merge Sort membagi array menjadi sub-array yang lebih kecil hingga setiap sub-array hanya berisi satu elemen. Setelah itu, sub-array tersebut digabungkan kembali dalam urutan yang tepat. Quick Sort juga menggunakan Divide and Conquer dengan memilih sebuah elemen sebagai pivot dan membagi array menjadi beberapa partisi yang lebih kecil.

Memahami rekursi dan pendekatan Divide and Conquer merupakan dasar penting sebelum mempelajari Merge Sort dan Quick Sort. Kedua algoritma tersebut menggunakan pemanggilan fungsi secara rekursif untuk membagi masalah besar menjadi submasalah dan memprosesnya secara efisien.

2. Merge Sort

2.1. Definisi Merge Sort

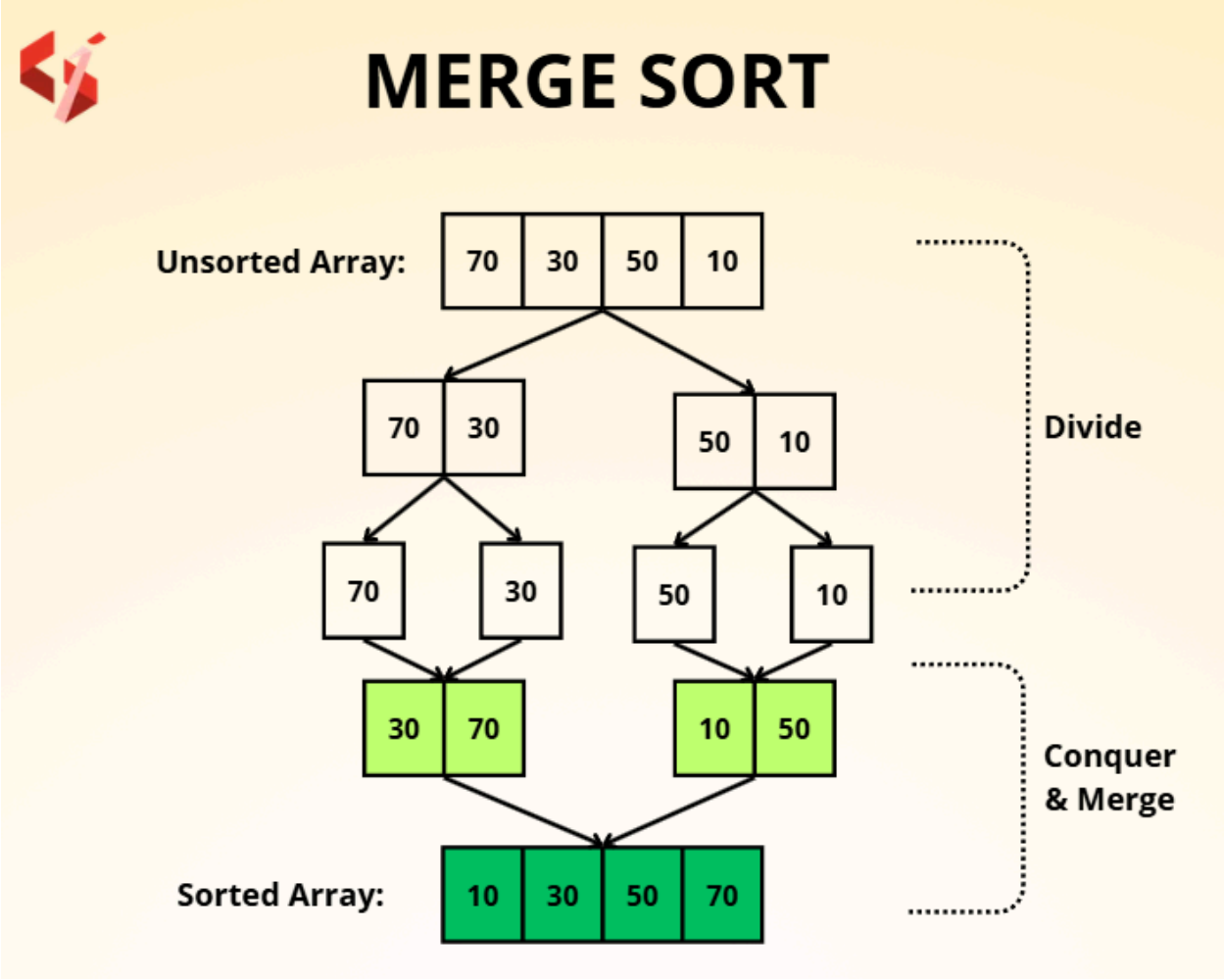
Merge Sort merupakan algoritma pengurutan yang menggunakan pendekatan Divide and Conquer untuk mengurutkan data secara efisien. Algoritma ini secara berulang membagi array menjadi sub-array yang lebih kecil hingga setiap sub-array hanya berisi satu elemen. Setelah

proses pembagian selesai, sub-array tersebut digabungkan kembali dalam urutan yang benar hingga menghasilkan array yang sepenuhnya terurut.

Merge Sort dianggap sebagai salah satu algoritma pengurutan berbasis perbandingan yang efisien karena kompleksitas waktunya tetap konsisten bahkan ketika digunakan pada dataset berukuran besar. Berbeda dengan beberapa algoritma pengurutan yang performanya bergantung pada urutan awal data, Merge Sort memiliki kompleksitas waktu O(n log n) pada best case, average case, dan worst case.

2.2. Cara Kerja Merge Sort

Merge Sort mengikuti paradigma Divide and Conquer yang terdiri dari tiga tahapan utama: **Divide**, **Conquer**, dan **Merge**. Tahapan-tahapan ini dilakukan secara rekursif hingga seluruh array terurut.

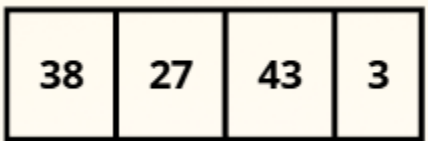


Gambar 3. *Ilustrasi Merge Sort*

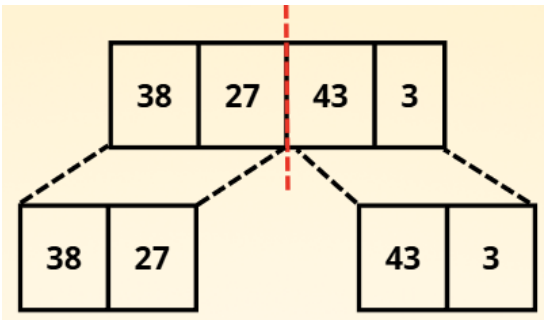
Divide

Tahap pertama adalah membagi array menjadi dua bagian yang lebih kecil. Setiap bagian kemudian terus dibagi hingga setiap sub-array hanya berisi satu elemen.

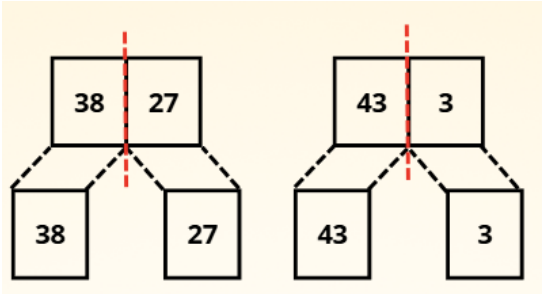
Sebagai contoh, perhatikan array berikut:



Array tersebut dibagi menjadi beberapa bagian sebagai berikut:



[38, 27, 43, 3] dibagi menjadi [38, 27] dan [43, 3]



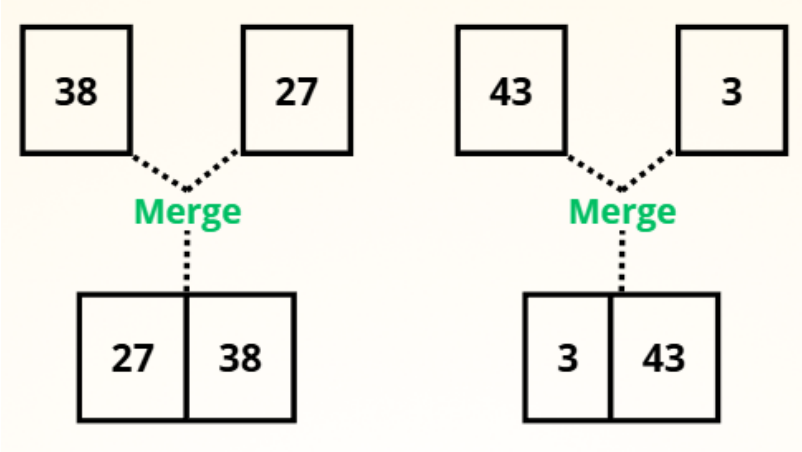
Kemudian:
[38, 27] dibagi menjadi [38] dan [27]
[43, 3] dibagi menjadi [43] dan [3]

Conquer

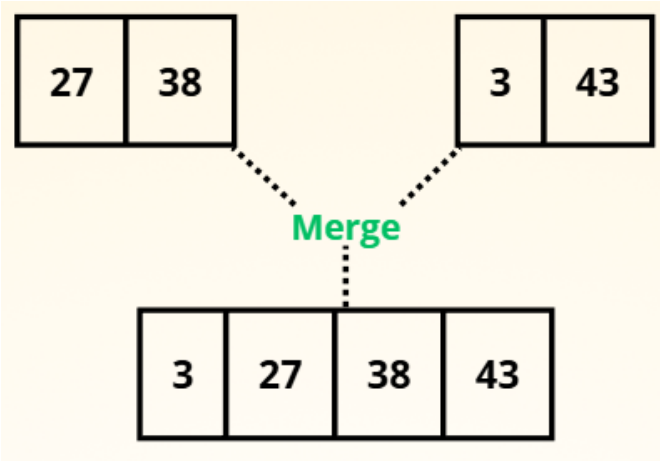
Setelah proses pembagian selesai, setiap sub-array diproses secara rekursif. Sub-array yang hanya berisi satu elemen sudah dianggap terurut, sehingga proses rekursif berhenti ketika tidak ada pembagian lebih lanjut yang dapat dilakukan. Tahap ini memastikan setiap sub-array mencapai bentuk paling sederhana sebelum digabungkan kembali. Pada Merge Sort, tahap Conquer secara implisit tercapai ketika setiap sub-array hanya berisi satu elemen, karena satu elemen sudah dianggap terurut.

Merge

Tahap terakhir adalah menggabungkan kembali sub-array sambil tetap mempertahankan urutan yang benar. Pada proses ini, elemen dari setiap sub-array dibandingkan dan disusun dari nilai terkecil hingga terbesar jika menggunakan pengurutan ascending.



Merge (gabungkan) [38] dan [27] untuk dapat [27, 38]
Merge (gabungkan) [43] dan [3] untuk dapat [3, 43]



Contoh Proses Merge:
Perhatikan sub-array yang sudah terurut berikut:
Left Array (kiri) = [27, 38]
Right Array (kanan) = [3, 43]
Algoritma membandingkan elemen pertama dari masing-masing sub-array untuk menentukan elemen yang ditempatkan terlebih dahulu ke dalam array hasil.

Langkah 1 (Perbandingan): 27 vs 3
Karena 3 lebih kecil, nilai 3 dimasukkan ke dalam array hasil.
Hasil = [3]
Elemen yang tersisa:
Kiri = [27, 38]
Kanan = [43]

Langkah 2 (Perbandingan): 27 vs 43
Karena 27 lebih kecil, nilai 27 dimasukkan ke dalam array hasil.
Hasil = [3, 27]
Elemen yang tersisa:
Kiri = [38]
Kanan = [43]

Langkah 3 (Perbandingan): 38 vs 43
Karena 38 lebih kecil, nilai 38 dimasukkan ke dalam array hasil.
Hasil = [3, 27, 38]
Elemen yang tersisa:
Kiri = [ ]
Kanan = [43]

Langkah 4:
Array sebelah kiri sudah kosong, sehingga elemen yang tersisa pada array sebelah kanan langsung disalin ke array hasil.
Hasil = [3, 27, 38, 43]
Proses merge terus dilakukan hingga seluruh elemen dari kedua sub-array telah disalin ke dalam array hasil.

Code Implementation

```
public class MergeSort {
   // Fungsi untuk menggabungkan dua sub-array yang sudah terurut
   public static void merge(int[] arr, int left, int mid, int right) {

       // Menghitung ukuran dari kedua sub-array
       int n1 = mid - left + 1;
       int n2 = right - mid;

       // Membuat array sementara
       int[] leftArray = new int[n1];
       int[] rightArray = new int[n2];

       // Menyalin data ke sub-array kiri
       for (int i = 0; i < n1; i++) {
           leftArray[i] = arr[left + i];
       }

       // Menyalin data ke sub-array kanan
       for (int j = 0; j < n2; j++) {
           rightArray[j] = arr[mid + 1 + j];
       }
```

```
        // Menentukan indeks awal untuk proses penggabungan
        int i = 0;
        int j = 0;
        int k = left;

      // Membandingkan elemen dan menggabungkannya dalam urutan terurut
        while (i < n1 && j < n2) {
            if (leftArray[i] <= rightArray[j]) {
                arr[k] = leftArray[i];
                i++;
            } else {
                arr[k] = rightArray[j];
                j++;
            }
            k++;
        }

        // Menyalin sisa elemen dari leftArray
        while (i < n1) {
            arr[k] = leftArray[i];
            i++;
            k++;
        }

        // Menyalin sisa elemen dari rightArray
        while (j < n2) {
            arr[k] = rightArray[j];
            j++;
            k++;
        }
    }
```

```
// Fungsi Merge Sort secara rekursif
public static void mergeSort(int[] arr, int left, int right) {

    // Base case: berhenti ketika sub-array hanya memiliki satu elemen
    if (left < right) {

        // Menentukan indeks tengah
        int mid = left + (right - left) / 2;
```

```
        // Membagi bagian kiri
        mergeSort(arr, left, mid);

        // Membagi bagian kanan
        mergeSort(arr, mid + 1, right);

        // Menggabungkan kedua bagian yang sudah terurut
        merge(arr, left, mid, right);
    }
}
```

```
public static void main(String[] args) {
        int[] data = {38, 27, 43, 3};
        mergeSort(data, 0, data.length - 1);
        System.out.print("Sorted Array: ");
        for (int value : data) {
            System.out.print(value + " ");
        }
    }
}
```

Output:

```
Sorted Array: 3 27 38 43
```

Masih Membutuhkan Penjelasan?
Jika kamu masih kesulitan memahami konsep dan alur kerja Merge Sort, kamu dapat menonton video berikut untuk mendapatkan penjelasan visual yang lebih mendetail:
Merge_Sort_Visualization_and_Explanation-VIDEO

Menonton proses pengurutan secara bertahap dapat membantu kamu memahami bagaimana tahapan Divide, Conquer, dan Merge bekerja bersama untuk menghasilkan array yang terurut.

2.3. Analisis Kompleksitas Waktu Merge Sort

Kompleksitas waktu digunakan untuk mengukur bagaimana waktu eksekusi suatu algoritma bertambah seiring dengan meningkatnya ukuran input. Merge Sort dianggap sebagai algoritma pengurutan yang efisien karena performanya tetap konsisten terlepas dari susunan awal data.

Dengan demikian, kompleksitas waktu Merge Sort secara keseluruhan adalah:
$T(n) = O(n \log n)$

Tabel 1. Kompleksitas Waktu Merge Sort

Case	Kompleksitas	Penjelasan
Best Case	O(n log n)	Terjadi ketika array dibagi menjadi bagian-bagian yang hampir sama besar pada setiap langkah rekursif. Proses pembagian menghasilkan log_n tingkat, sedangkan proses penggabungan membutuhkan n operasi pada setiap tingkat.
Average Case	O(n log n)	Pada sebagian besar kondisi, Merge Sort membagi array menjadi sub-array yang seimbang dan melakukan proses penggabungan secara efisien, sehingga menghasilkan kompleksitas O(n log n).
Worst Case	O(n log n)	Bahkan dalam kondisi terburuk, Merge Sort tetap membagi array menjadi bagian yang sama besar dan memproses seluruh elemen selama proses penggabungan. Oleh karena itu, kompleksitasnya tetap O(n log n).

Salah satu keunggulan utama Merge Sort adalah performanya yang konsisten. Berbeda dengan beberapa algoritma pengurutan lainnya, Merge Sort memiliki kompleksitas waktu yang sama pada kondisi terbaik, rata-rata, maupun terburuk.

Selain itu, Merge Sort membutuhkan memori tambahan selama proses penggabungan karena menggunakan array sementara untuk menyimpan sub-array sebelum digabungkan. Oleh karena itu, kompleksitas ruang (space complexity) Merge Sort adalah O(n).

2.4. Kelebihan dan Kekurangan Merge Sort

Seperti algoritma pengurutan lainnya, Merge Sort memiliki kelebihan dan kekurangan. Memahami karakteristik tersebut dapat membantu programmer menentukan apakah Merge Sort sesuai untuk suatu permasalahan atau aplikasi tertentu.

Kelebihan

- Efisiensi untuk Dataset Berukuran Besar
  Merge Sort memiliki kompleksitas waktu O(n log n), sehingga cocok untuk mengurutkan data dalam jumlah besar secara efisien.
- Performa yang konsisten
  Berbeda dengan beberapa algoritma pengurutan yang performanya bergantung pada susunan awal data, Merge Sort memiliki kompleksitas waktu yang sama pada kondisi terbaik, rata-rata, dan terburuk.
- Algoritma Pengurutan yang Stabil

Merge Sort mempertahankan urutan relatif elemen yang memiliki nilai yang sama, sehingga cocok digunakan untuk mengurutkan data yang memiliki beberapa atribut.

- Cocok untuk Linked List dan External Sorting
  Merge Sort dapat bekerja secara efisien pada linked list dan sering digunakan dalam external sorting ketika seluruh data tidak dapat dimuat ke dalam memori sekaligus.

Kekurangan

- Membutuhkan Memori Tambahan
  Merge Sort menggunakan array sementara selama proses penggabungan, sehingga membutuhkan memori tambahan.
- Bukan Algoritma In-Place
  Karena membutuhkan ruang penyimpanan tambahan, Merge Sort umumnya kurang efisien dalam penggunaan memori dibandingkan dengan algoritma in-place seperti Quick Sort.
- Overhead Rekursif
  Sifat rekursif Merge Sort menghasilkan pemanggilan fungsi tambahan yang dapat meningkatkan overhead eksekusi dibandingkan dengan beberapa algoritma pengurutan yang lebih sederhana.

Dengan performanya yang konsisten dan stabilitasnya, Merge Sort sering dipilih untuk aplikasi yang membutuhkan hasil pengurutan yang andal, terutama saat bekerja dengan dataset berukuran besar. Namun, kebutuhan memori tambahan perlu dipertimbangkan ketika efisiensi penggunaan memori menjadi prioritas.

3. Quick Sort

3.1. Definisi Quick Sort

Quick Sort merupakan algoritma pengurutan yang menggunakan pendekatan **Divide** and **Conquer** untuk mengorganisasi data secara efisien. Algoritma ini bekerja dengan memilih sebuah elemen yang disebut pivot, kemudian membagi array menjadi dua kelompok: elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Proses yang sama kemudian diterapkan secara rekursif pada setiap partisi hingga seluruh array terurut.

Berbeda dengan Merge Sort yang membagi array menjadi dua bagian yang relatif sama besar, Quick Sort berfokus pada menempatkan elemen pivot pada posisi yang tepat dan mengatur elemen lainnya di sekitar pivot tersebut. Setelah pivot berada pada posisi yang benar, algoritma secara rekursif mengurutkan partisi di sebelah kiri dan di sebelah kanan pivot.

Quick Sort banyak digunakan karena memiliki performa yang baik dalam praktik dan penggunaan memori yang relatif rendah. Karena sebagian besar operasi dilakukan langsung pada array asli, Quick Sort termasuk algoritma pengurutan in-place. Karakteristik ini

membuatnya menjadi pilihan populer dalam berbagai aplikasi perangkat lunak dan perpustakaan.

Performa Quick Sort sangat dipengaruhi oleh pemilihan elemen sebagai pivot. Oleh karena itu, pemilihan pivot yang tepat penting untuk memperoleh proses pengurutan yang lebih efisien.

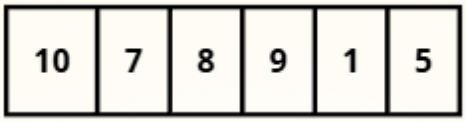
3.2. Cara Kerja Quick Sort

Quick Sort mengikuti paradigma Divide and Conquer dengan memilih sebuah elemen sebagai pivot dan membagi array berdasarkan pivot tersebut. Algoritma kemudian menempatkan pivot pada posisi yang tepat dan secara rekursif mengurutkan partisi di sebelah kiri dan kanan hingga seluruh array terurut.

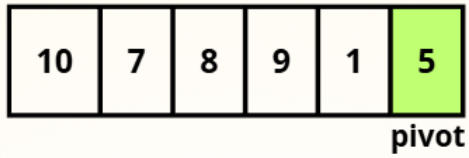
Konsep Pivot

Pivot merupakan elemen yang dipilih sebagai acuan dalam proses partitioning. Pemilihan pivot dapat bervariasi, misalnya elemen pertama, elemen terakhir, elemen secara acak, atau elemen tengah. Elemen yang lebih kecil dari pivot akan ditempatkan di sebelah kiri, sedangkan elemen yang lebih besar dari pivot akan ditempatkan di sebelah kanan.

Sebagai contoh, perhatikan array berikut:

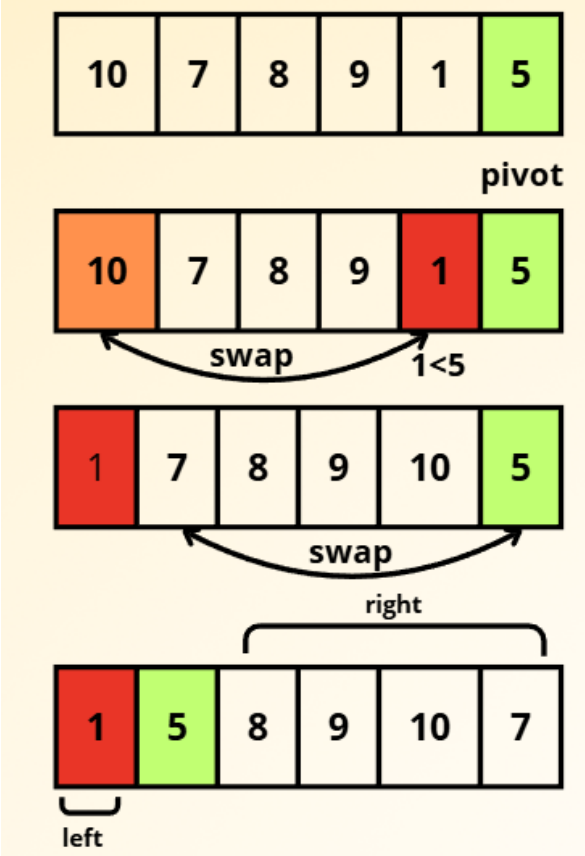


Pada contoh ini, elemen terakhir, yaitu 5, dipilih sebagai pivot.



Partition

Setelah memilih pivot, algoritma mengatur ulang elemen sehingga semua nilai yang lebih kecil dari pivot ditempatkan di sebelah kiri, sedangkan nilai yang lebih besar dari pivot ditempatkan di sebelah kanan. Algoritma membandingkan setiap elemen dengan nilai pivot untuk menentukan posisinya.

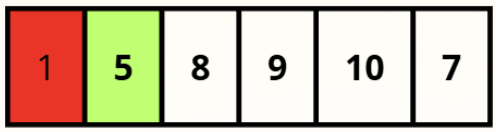


Pada tahap ini, elemen di sebelah kanan pivot belum sepenuhnya terurut. Proses partisi hanya memastikan bahwa pivot berada pada posisi yang tepat. Partisi di sebelah kiri dan kanan kemudian akan diproses secara rekursif pada tahap berikutnya.

Recursive Sorting

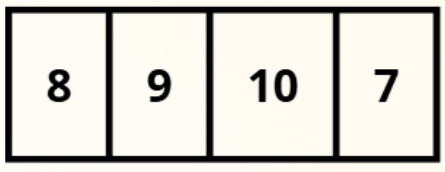
Setelah pivot ditempatkan pada posisi yang benar, Quick Sort secara rekursif menerapkan proses yang sama pada partisi di sebelah kiri dan kanan pivot. Setiap partisi akan memilih pivot-nya sendiri, melakukan proses partisi, lalu dilanjutkan secara rekursif hingga setiap partisi hanya memiliki satu elemen.

Perhatikan hasil dari proses partisi sebelumnya:

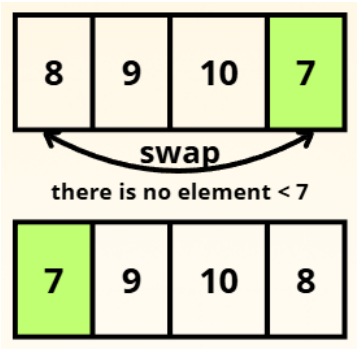


Pivot (5) sudah berada pada posisi yang tepat dalam urutan. Oleh karena itu, Quick Sort hanya perlu memproses partisi yang tersisa.

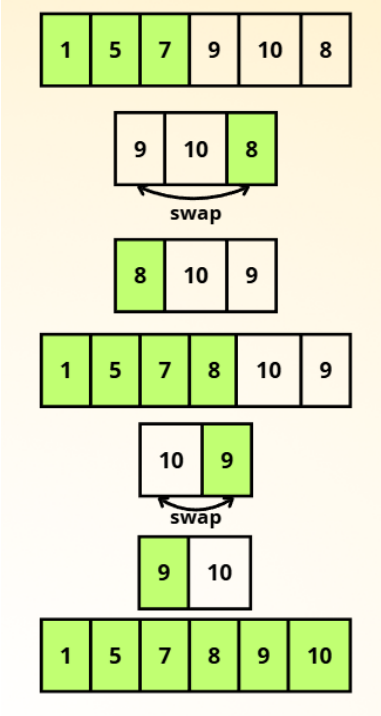
Partisi sebelah kiri hanya memiliki satu elemen: [1]
Karena partisi yang hanya memiliki satu elemen sudah dianggap terurut, tidak diperlukan proses lebih lanjut.
Partisi sebelah kanan adalah:



Quick Sort kembali diterapkan pada partisi tersebut. Misalkan elemen terakhir (7) dipilih sebagai pivot baru.



Pivot (7) sekarang berada pada posisi yang tepat. Proses kemudian dilanjutkan secara rekursif pada partisi yang tersisa hingga seluruh elemen terurut.



Proses rekursif berhenti ketika setiap partisi hanya berisi satu elemen. Pada titik ini, seluruh array telah terurut.

Implementasi Kode

```
public class QuickSort {
   // Fungsi untuk membagi array berdasarkan pivot
   public static int partition(int[] arr, int low, int high) {
     // Memilih elemen terakhir sebagai pivot (dalam contoh ini)
       int pivot = arr[high];
       // Indeks untuk elemen yang lebih kecil dari pivot
       int i = low - 1;
       // Membandingkan setiap elemen dengan pivot
       for (int j = low; j < high; j++) {
           if (arr[j] < pivot) {
               i++;
               // Menukar posisi elemen
               int temp = arr[i];
               arr[i] = arr[j];
               arr[j] = temp;
           }
       }
       // Menempatkan pivot pada posisi yang benar
       int temp = arr[i + 1];
```

```
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }

    // Fungsi Quick Sort secara rekursif
    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            // Membagi array dan mendapatkan indeks pivot
            int pivotIndex = partition(arr, low, high);
            // Mengurutkan partisi sebelah kiri
            quickSort(arr, low, pivotIndex - 1);
            // Mengurutkan partisi sebelah kanan
            quickSort(arr, pivotIndex + 1, high);
        }
    }


    public static void main(String[] args) {
        int[] data = {10, 7, 8, 9, 1, 5};
        // Mengurutkan array menggunakan Quick Sort
        quickSort(data, 0, data.length - 1);
        System.out.print("Sorted Array: ");
        for (int value : data) {
            System.out.print(value + " ");
        }
    }
}
```

Output

```
Sorted Array: 1 5 7 8 9 10
```

Jika kamu masih kesulitan memahami konsep dan alur kerja Quick Sort, kamu dapat menonton video berikut untuk mendapatkan penjelasan visual yang lebih mendetail:
Quick_Sort_Visualization-VIDEO
Menonton proses partitioning dan pengurutan secara rekursif secara bertahap dapat membantu kamu memahami bagaimana Quick Sort menggunakan pivot untuk mengurutkan array secara efisien.

3.3. Analisis Kompleksitas Waktu Quick Sort

Kompleksitas waktu digunakan untuk mengukur bagaimana waktu eksekusi suatu algoritma bertambah seiring dengan meningkatnya ukuran input. Performa Quick Sort sangat bergantung pada bagaimana elemen pivot membagi array pada setiap proses partitioning. Oleh karena itu, kompleksitas waktu Quick Sort dapat berbeda tergantung pada kualitas proses pembagiannya.

Tabel 2. *Kompleksitas Waktu Quick Sort*

Case	Kompleksitas	Penjelasan
Best Case	O(n log n)	Terjadi ketika pivot membagi array menjadi dua partisi yang hampir sama besar pada setiap langkah rekursif. Hal ini menghasilkan log_n tingkat log_n, dengan n operasi yang dilakukan pada setiap tingkat.
Average Case	O(n log n)	Pada sebagian besar kondisi, pivot menghasilkan partisi yang cukup seimbang sehingga Quick Sort dapat mempertahankan performa yang efisien.
Worst Case	$O(n^2)$	Terjadi ketika pivot dilakukan secara berulang sehingga menghasilkan partisi yang sangat tidak seimbang, misalnya ketika elemen terkecil atau terbesar selalu dipilih sebagai pivot.

Salah satu faktor yang membuat Quick Sort efisien dalam praktik adalah kemampuannya mengurutkan data tanpa memerlukan memori tambahan yang besar. Karena sebagian besar operasi dilakukan langsung pada array asli, penggunaan memori Quick Sort umumnya lebih rendah dibandingkan dengan Merge Sort.

Space Complexity: O(log n)

Meskipun Quick Sort dapat memiliki kompleksitas waktu $O(n^2)$ pada kondisi terburuk, algoritma ini sering kali lebih cepat dibandingkan algoritma pengurutan lainnya dalam penggunaan nyata karena strategi partitioning yang efisien dan penggunaan memori yang rendah.

3.4. Kelebihan dan Kekurangan Quick Sort

Seperti algoritma pengurutan lainnya, Quick Sort memiliki kelebihan dan kekurangan. Memahami karakteristik tersebut dapat membantu programmer menentukan apakah Quick Sort sesuai untuk suatu permasalahan atau aplikasi tertentu.

Kelebihan:

- Efisien dalam Praktik
  Quick Sort merupakan salah satu algoritma pengurutan yang cepat dalam banyak aplikasi nyata karena strategi partitioning-nya yang efisien.
- Algoritma In-Place
  Sebagian besar operasi dilakukan langsung pada array asli, sehingga penggunaan memori tambahan relatif rendah
- Performa Average Case yang Baik
  Quick Sort memiliki kompleksitas waktu rata-rata O(n log n), sehingga efisien untuk mengurutkan dataset berukuran besar.
- Banyak Digunakan dalam Aplikasi Perangkat Lunak
  Karena kecepatan dan kesederhanaannya, Quick Sort banyak digunakan dalam pustaka pemrograman dan berbagai sistem perangkat lunak.

Kekurangan:

- Kompleksitas Waktu Terburuk $O(n^2)$
  Jika pemilihan pivot yang buruk terjadi berulang kali, algoritma dapat menghasilkan partisi yang sangat tidak seimbang sehingga menyebabkan performa yang lebih lambat.
- Performa Bergantung pada Pemilihan Pivot
  Efisiensi Quick Sort sangat dipengaruhi oleh cara pivot dipilih pada setiap proses partitioning.
- Bukan Algoritma Pengurutan yang Stabil
  Elemen dengan nilai yang sama tidak selalu mempertahankan urutan relatifnya setelah proses pengurutan
- Overhead Rekursif
  Seperti algoritma rekursif lainnya, Quick Sort membutuhkan pemanggilan fungsi tambahan selama proses eksekusi.

Quick Sort sering dipilih ketika efisiensi memori dan performa menjadi prioritas dalam praktik. Namun, pengaruh pemilihan pivot perlu dipertimbangkan karena dapat berdampak signifikan pada kinerja keseluruhan algoritma.

Tabel 3. *Perbandingan Merge Sort dan Quick Sort*

Aspek	Merge Sort	Quick Sort
Pendekatan	Divide and Conquer	Divide and Conquer
Kompleksitas Waktu (Rata-rata)	O(n log n)	O(n log n)
Worst Case	O(n log n)	$O(n^2)$
Kompleksitas Ruang	O(n)	O(log n)
Stabilitas	Stabil	Tidak Stabil
Penggunaan Memory	Lebih tunggi	Lebih Rendah
In-Place	Tidak	Ya

Merge Sort memberikan performa yang konsisten dan pengurutan yang stabil, sedangkan Quick Sort umumnya lebih cepat dalam praktik dan membutuhkan memori yang lebih sedikit. Pemilihan antara kedua algoritma tersebut bergantung pada kebutuhan spesifik aplikasi.

CODELAB

Task 1 - Merge Sort (Descending Order)

Objective

Pada tugas ini, kamu akan melengkapi bagian yang kosong dalam implementasi Merge Sort untuk mengurutkan data secara menurun. Lengkapi semua bagian yang kosong, lalu jalankan program hingga menghasilkan output yang diharapkan.

```java
public class MergeSortDescending {
    public static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int[] leftArray = new int[n1];
        int[] rightArray = new int[n2];
        for (int i = 0; i < n1; i++) {
            leftArray[i] = arr[left + i];
        }
        for (int j = 0; j < n2; j++) {
            rightArray[j] = arr[mid + 1 + j];
        }
        int i = 0;
        int j = 0;
        int k = left;
        // TODO: Isi operator perbandingan untuk urutan descending
        while (i < n1 && j < n2) {
            if (leftArray[i] _____ rightArray[j]) {
                arr[k] = leftArray[i];
                i++;
            } else {
                arr[k] = rightArray[j];
                j++;
            }
            k++;
        }
        while (i < n1) {
            arr[k] = leftArray[i];
            i++;
            k++;
        }
        while (j < n2) {
            arr[k] = rightArray[j];
            j++;
            k++;
        }
```

```java
public static void mergeSort(int[] arr, int left, int right) {

    // TODO: Isi kondisi base case
    if ( _____ ) {

        // TODO: Hitung indeks tengah
        int mid = _____;

        // TODO: Pemanggilan rekursif untuk sub-array kiri
        _____;

        // TODO: Pemanggilan rekursif untuk sub-array kanan
        _____;

        merge(arr, left, mid, right);
    }
}
```

```java
public static void main(String[] args) {

    int[] data = {38, 27, 43, 3};

    mergeSort(data, 0, data.length - 1);

    System.out.print("Sorted Array (Descending): ");

    for (int value : data) {
        System.out.print(value + " ");
    }
}
}
```

Expected Output

```
Sorted Array (Descending): 43 38 27 3
```

Sebelum meminta bantuan, pastikan kamu dapat menjawab pertanyaan berikut:

1. Kondisi apa yang digunakan untuk menghentikan proses rekursif pada Merge Sort?
2. Bagaimana cara menghitung indeks tengah (middle index)?
3. Mengapa Merge Sort melakukan proses rekursif pada sub-array kiri dan kanan?
4. Operator perbandingan apa yang digunakan untuk menghasilkan urutan descending?
5. Apa fungsi metode merge() dalam algoritma Merge Sort?

Task 2 - Quick Sort (Descending Order)

Objective

Pada tugas ini, kamu akan melengkapi bagian yang kosong dalam implementasi Quick Sort untuk mengurutkan data secara menurun. Lengkapi semua bagian yang kosong, lalu jalankan program hingga menghasilkan output yang diharapkan.

```java
public class QuickSortDescending {
    public static int partition(int[] arr, int low, int high) {
        // TODO: Pilih elemen pivot
        int pivot = _____;
        int i = low - 1;
        for (int j = low; j < high; j++) {
            // TODO: Isi operator perbandingan untuk urutan descending
            if (arr[j] _____ pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }

    public static void quickSort(int[] arr, int low, int high) {
        // TODO: Isi kondisi base case
        if ( _____ ) {
            // TODO: Lakukan partitioning dan simpan indeks pivot
            int pivotIndex = _____;
            quickSort(arr, low, pivotIndex - 1);
            quickSort(arr, pivotIndex + 1, high);
        }
    }
    public static void main(String[] args) {
        int[] data = {10, 7, 8, 9, 1, 5};
        // TODO: Pemanggilan awal Quick Sort
        _____;
        System.out.print("Sorted Array (Descending): ");

        for (int value : data) {
            System.out.print(value + " ");
        }
    }
}
```

Expected output:

```
Sorted Array (Descending): 10 9 8 7 5 1
```

Sebelum meminta bantuan, pastikan kamu dapat menjawab pertanyaan berikut:

1. Apa fungsi pivot dalam Quick Sort?
2. Mengapa proses partitioning penting dalam Quick Sort?
3. Operator perbandingan apa yang digunakan untuk menghasilkan urutan descending?
4. Kondisi apa yang digunakan untuk menghentikan proses rekursif?
5. Mengapa Quick Sort melakukan proses rekursif pada partisi kiri dan kanan?

DEMO

Z-App: Building a Recommendation & Digital Trend Engine

Project Overview

Demo ini melanjutkan pengembangan platform Z-App dari modul-modul sebelumnya. Pada Modul 1, Z-App mengimplementasikan Bubble Sort dan Selection Sort untuk mengatur peringkat influencer berdasarkan jumlah pengikut. Pada Modul 2, platform dikembangkan dengan menambahkan Insertion Sort dan Shell Sort untuk meningkatkan performa pengurutan pada dataset berukuran sedang.

Seiring dengan perkembangan platform, jumlah influencer dan digital creator meningkat secara signifikan. Sistem peringkat kini perlu memproses dataset yang lebih besar dengan lebih efisien sambil mengekalkan masa tindak balas yang pantas. Untuk mengatasi tantangan tersebut, tim pengembang memutuskan untuk meningkatkan mesin peringkat dengan mengimplementasikan algoritma pengurutan rekursif yang lebih lanjut:

- Merge Sort
- Quick Sort

Pada modul ini, mahasiswa akan mengembangkan sistem Z-App dengan algoritma pengurutan rekursif dan menganalisis performanya pada dataset berukuran besar.

Scenario & Requirements

Kamu masih menjadi bagian dari tim pengembang Z-App.

Fitur "Top 100 Influencers" yang diperkenalkan pada modul sebelumnya telah menjadi salah satu fitur paling populer di platform ini. Seiring meningkatnya jumlah influencer yang perlu diproses, sistem peringkat kini membutuhkan kemampuan untuk menangani dataset yang lebih besar secara lebih efisien.

Untuk meningkatkan sistem peringkat, tim pengembang memutuskan untuk mengintegrasikan Merge Sort dan Quick Sort sebagai metode pengurutan alternatif. Pengguna harus dapat membandingkan berbagai algoritma pengurutan dan memilih algoritma yang paling sesuai dengan kebutuhan mereka.

Mahasiswa diminta untuk mengimplementasikan:

- Merge Sort
- Quick Sort

Sistem harus mendukung:

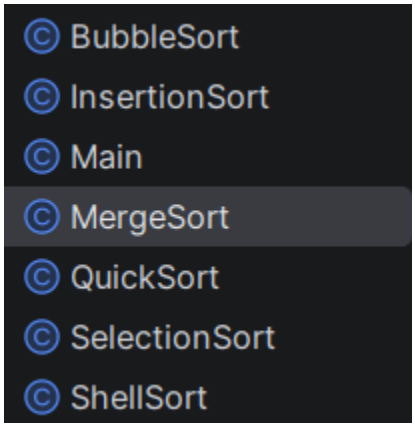
- Urutan ascending (kecil ke besar)
- Urutan Descending (besar ke kecil)

Proses pengurutan juga harus menampilkan tahapan rekursif atau proses partitioning untuk membantu pengguna memahami bagaimana algoritma bekerja secara internal.

Tasks

1. Buat class MergeSort.java
   Class ini harus memiliki:
   - Metode Merge Sort untuk urutan ascending
   - Metode Merge Sort untuk urutan descending
   - Visualisasi proses divide dan merge
2. Buat class QuickSort.java
   Class ini harus memiliki:
   - Metode Quick Sort untuk urutan ascending
   - Metode Quick Sort untuk urutan descending
   - Visualisasi proses partitioning
   - Informasi pivot pada setiap tahap rekursif
3. Perbarui class Main.java
   - Pertahankan seluruh fitur pengurutan dari modul sebelumnya
   - Memungkinkan pengguna memilih algoritma pengurutan
   - Memungkinkan pengguna memilih urutan ascending atau descending
   - Display:
     - Array awal
     - Proses pengurutan
     - Hasil akhir pengurutan

Structure File:



Notes:

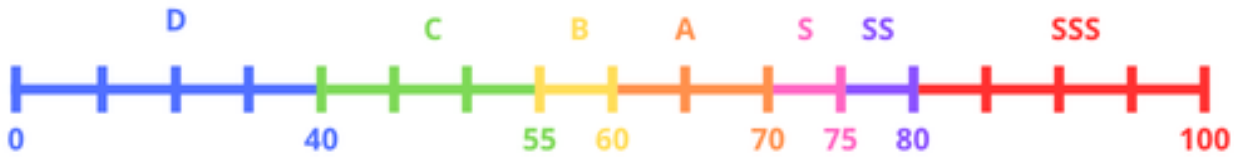
- Jangan menggunakan metode pengurutan bawaan seperti Arrays.sort().
- Program harus diimplementasikan secara manual.
- Metode rekursif harus diimplementasikan dengan benar.
- Pastikan proses divide, merge, dan partition ditampilkan dengan jelas.
- Pertahankan kompatibilitas dengan sistem yang telah dikembangkan pada Modul 1 dan Modul 2.

Penilaian

Rubrik Penilaian

Aspek Penilaian	Point
Codelab	**Total 20%**
DEMO	**Total 80%**
Program Benar	10%
Algorithm Implementation	40%
Answer Questions	30%

Rating Scale



SSS = (81 - 100) → SSSensational!!!

SS = (75 - 80) → Sick Skills!!

S = (70 - 74) → Sweet!

A = (60 - 69) → Alright

B = (55 - 59) → Bravo!

C = (41 - 54) → Come on!

D = (0 - 40) → Dull